

HireIQ

AI Hiring Intelligence Platform

ML Specification Document (MLSD)

Document Type	ML Specification Document (MLSD)
Product	HireIQ — AI Hiring Intelligence Platform
Version	v1.0.0
Parent Documents	HireIQ PRD v1.0 HireIQ TAD v1.0 HireIQ BSD v1.0
Status	DRAFT — ML Engineering Review
Author	ML Engineering Team
Date	June 15, 2026
Primary Stack	Python 3.11 spaCy scikit-learn Sentence-Transformers FastAPI

SECTION 1

ML System Overview & Design Philosophy

1. ML System Overview & Design Philosophy

This document is the complete specification for the HireIQ Machine Learning system. It defines the architecture, data requirements, model design, training procedures, evaluation benchmarks, inference pipelines, deployment strategy, and monitoring requirements for all four AI engines that power the platform.

DESIGN MANDATE

All ML models in HireIQ v1.0 must be explainable, deterministic, and auditable. Black-box deep learning models with no interpretability are not acceptable for candidate scoring — every score must have a traceable reason. This is both an ethical requirement and a product requirement.

1.1 ML System Components

Engine 1 — Resume Intelligence	NLP pipeline that converts raw PDF text into structured JSON: skills, experience, education, projects. The foundational engine that feeds all other systems.
Engine 2 — Candidate Scoring	Semantic similarity engine that scores a candidate (0-100) against a job description. Uses TF-IDF vectorization and cosine similarity for explainable, deterministic scoring.
Engine 3 — Job Recommendation	Content-based recommendation system that matches a user skill profile against all active job postings. Returns top 5 personalized job recommendations.
Engine 4 — Interview Evaluation	Answer evaluation system using sentence embeddings to score candidate interview responses against ideal answers. Returns score, feedback, and confidence level.

1.2 Guiding ML Principles

Explainability First	Every AI score must be accompanied by reasons: matched_skills, missing_skills, and feedback text. No score without explanation.
Determinism	Given the same input, the system must always return the same output. No random sampling or stochastic elements in inference pipelines.
Graceful Degradation	If any ML engine fails, the platform must continue to function. Failed AI scores are stored as null — the application still succeeds.

Performance SLA	All inference endpoints must respond within 5 seconds at p95. Model loading happens at startup, never per-request.
Bias Awareness	Scoring is based on skill and semantic match only. No demographic signals (name, location, graduation year) are used in scoring features.
Iterative Improvement	Models are versioned. Each model version has a benchmark score. A new version is only deployed if it improves on the benchmark.

1.3 ML Service Architecture

```

Core API (FastAPI, Port 8000)
|
| HTTP POST (resume_text, job_description)
| Internal VPC network – not public
v
AI Service (FastAPI, Port 8001)
+-----+
| Model Loader (startup) |
|   loads: spaCy model, TF-IDF vectorizer, |
|           job matrix, sentence transformer |
| |
| /analyze-resume --> Resume Intelligence |
| /score-candidate --> Candidate Scoring |
| /recommend-jobs --> Job Recommendation |
| /evaluate-interview -> Interview Evaluation |
+-----+
|
| Redis (model cache, job matrix cache)
v
AWS ElastiCache Redis (Port 6379)

```

SECTION 2

Engine 1 — Resume Intelligence

2. Engine 1 — Resume Intelligence Engine

The Resume Intelligence Engine is the entry point of the entire HireIQ AI system. It converts unstructured PDF text into structured, machine-readable JSON that all downstream engines consume. This engine runs every time a user uploads a resume.

2.1 Input / Output Contract

Input	Plain text string extracted from a PDF resume. Typically 500–3,000 words. May contain formatting noise, section headers, dates, URLs, and special characters.
Output	Structured JSON: { skills: string[], experience: string, education: string, projects: string[], raw_entity_count: int, confidence: float }
Latency SLA	< 2 seconds per resume on standard CPU hardware
Accuracy Target	Skill extraction recall $\geq$ 85% on the 50-resume benchmark test set

2.2 Full NLP Pipeline

```
RAW PDF TEXT (string)
  |
  v
Step 1: Text Cleaning
  - Remove URLs (regex: https?://S+)
  - Remove email addresses (regex: [w.-]+@[w.-]+)
  - Remove phone numbers (regex: [+]?([]?[0-9]{3}[])?[-s.]?[0-9]{3}[-s.]?[0-9]{4,6})
  - Normalise whitespace (collapse multiple spaces and newlines)
  - Lowercase for matching (keep original for display)
  |
  v
Step 2: Section Detection
  - Identify section headers using keyword matching:
    SKILLS, EXPERIENCE, EDUCATION, PROJECTS, CERTIFICATIONS, SUMMARY
  - Split text into named sections (dict: section_name -> text_block)
  |
  v
Step 3: spaCy NLP Processing
  - Load en_core_web_lg model (once at startup)
```

```

- Run full NLP pipeline: tokenization, POS tagging, NER
- Extract entities: ORG (companies, schools), DATE, PERSON
  |
  v
Step 4: Skill Extraction (primary)
- Exact match: check every token/bigram/trigram against
skills_dictionary.json
- Fuzzy match: rapidfuzz.process.extract() with threshold >= 88
- Context match: skills found in SKILLS section given 2x weight
- Deduplicate: normalise (Python == python == PYTHON) and dedupe
  |
  v
Step 5: Experience Extraction
- Date range regex: (Jan|Feb|...|Dec)?s*(d{4})s*[--]s*(d{4}|Present)
- Duration phrases: (d+)s*+?s*(year|yr) of experience
- Calculate total: sum non-overlapping date ranges in years
- Output: string like 3 years or 5+ years
  |
  v
Step 6: Education Extraction
- Keyword list: B.Tech, B.E., B.Sc, M.Tech, M.Sc, MBA, PhD, BCA, MCA
- spaCy ORG entities cross-referenced with university keyword list
- Return highest qualification found
  |
  v
Step 7: Project Extraction
- Detect PROJECTS section header
- Extract project names: usually first sentence or bold text in each
block
- Extract tech stack: skills found within 3 lines of project name
  |
  v
OUTPUT JSON
{
  skills: [python, machine learning, docker, react, ...],
  experience: 3 years,
  education: B.Tech Computer Science,
  projects: [AI Chatbot, E-Commerce Platform],
  confidence: 0.87
}

```

2.3 Skills Dictionary

Size	2,000+ curated skill entries covering programming languages, frameworks, tools, methodologies, and soft skills.
Format	JSON array of objects: { name: string, aliases: string[], category: string, weight: float }
Categories	Programming Languages, Web Frameworks, ML/AI, Cloud/DevOps, Databases, Mobile, Testing, Soft Skills, Methodologies

Aliases	Each skill has aliases: Python aliases include [python3, py, python 3]. Matching is case-insensitive on aliases.
Weight	Skills in high-demand categories (ML/AI, Cloud) have weight > 1.0 for scoring boosts. Range: 0.5–2.0.
Maintenance	Skills dictionary versioned in Git. New skills added monthly based on job posting analysis. Alias expansion via community PRs.

2.4 Implementation

```
# app/engines/resume_engine.py
import spacy, json, re
from rapidfuzz import process, fuzz
from pathlib import Path

nlp = spacy.load('en_core_web_lg') # Loaded once at module import

with open('app/data/skills_dictionary.json') as f:
    SKILLS_DICT = json.load(f)

# Build flat lookup: alias -> canonical_skill_name
SKILL_LOOKUP = {}
for skill in SKILLS_DICT:
    SKILL_LOOKUP[skill['name'].lower()] = skill['name']
    for alias in skill.get('aliases', []):
        SKILL_LOOKUP[alias.lower()] = skill['name']

SKILL_KEYS = list(SKILL_LOOKUP.keys()) # For fuzzy matching

def clean_text(text: str) -> str:
    text = re.sub(r'https?://S+', '', text) # Remove URLs
    text = re.sub(r'[w.-]+@[w.-]+.w+', '', text) # Remove emails
    text = re.sub(r'[+]?[()?[0-9]{3}[]]?[-s]?[0-9]{3}[-s]?[0-9]{4,6}', '', text)
    return re.sub(r's+', ' ', text).strip()

def extract_skills(text: str) -> list[str]:
    text_lower = text.lower()
    found = set()

    # Exact match on tokens and bigrams
    tokens = text_lower.split()
    for n in range(1, 4): # unigram, bigram, trigram
        for i in range(len(tokens) - n + 1):
            ngram = ' '.join(tokens[i:i+n])
            if ngram in SKILL_LOOKUP:
                found.add(SKILL_LOOKUP[ngram])

    # Fuzzy match for near-matches (typos, variations)
    results = process.extract(text_lower, SKILL_KEYS, scorer=fuzz.partial_ratio,
                              limit=50, score_cutoff=88)
    for match, score, _ in results:
        found.add(SKILL_LOOKUP[match])
```

```

        return sorted(list(found))

def extract_experience(text: str) -> str:
    # Date range pattern: 2019 - 2022, Jan 2020 - Present
    ranges = re.findall(
        r'(d{4})s*[--]s*(d{4}|present|current)',
        text.lower()
    )
    total_years = 0
    for start, end in ranges:
        s = int(start)
        e = 2024 if end in ('present', 'current') else int(end)
        if 1990 <= s <= 2024 and e >= s:
            total_years += (e - s)

    # Direct mention pattern
    direct = re.findall(r'(d+)?s*year', text.lower())
    if direct and not ranges:
        total_years = max(int(d) for d in direct)

    if total_years == 0: return None
    return f'{total_years}+ years' if total_years >= 5 else f'{total_years} years'

def analyze_resume(resume_text: str) -> dict:
    cleaned = clean_text(resume_text)
    skills = extract_skills(cleaned)
    exp = extract_experience(cleaned)
    doc = nlp(cleaned[:100000]) # spaCy token limit guard
    edu = extract_education(doc)
    projects = extract_projects(cleaned)
    confidence = min(1.0, len(skills) / 10) # Simple confidence metric
    return {'skills':skills, 'experience':exp, 'education':edu,
            'projects':projects, 'confidence':round(confidence,2)}

```

2.5 Evaluation Benchmark

Metric	Definition	Target
Skill Recall	% of actual skills in resume that the engine correctly identifies	Target >= 85% on 50-resume benchmark set
Skill Precision	% of extracted skills that are genuinely present in the resume	Target >= 90% (few false positives)
Experience Accuracy	% of resumes where extracted experience matches human label within 1 year	Target >= 80%
Education Accuracy	% of resumes where education level correctly identified	Target >= 90%
End-to-End Latency	Time from text input to JSON output	Target < 2 seconds on 4-core CPU

Benchmark Dataset

50 manually annotated resumes
covering: fresh grad, 2yr, 5yr, 10yr+
experience levels

Maintained in
tests/data/resume_benchmark/

SECTION 3

Engine 2 — Candidate Scoring System

3. Engine 2 — Candidate Scoring System

The Candidate Scoring System is the core differentiator of HireIQ for recruiters. It produces an objective, explainable 0-100 match score between a candidate's resume and a job description, along with a skill gap breakdown. Every recruiter decision is informed by this score.

3.1 Input / Output Contract

Input	{ resume_text: string, job_description: string } — both raw text strings
Output	{ score: int (0-100), matched_skills: string[], missing_skills: string[], score_breakdown: object }
Latency SLA	< 500ms per scoring request (model loaded at startup)
Accuracy Target	Pearson correlation ≥ 0.80 with human expert scores on 30-pair test set

3.2 Scoring Algorithm

```
INPUTS: resume_text (R), job_description (J)
      |
      v
Step 1: Text Preprocessing
- Clean both texts: lowercase, remove punctuation, remove stopwords
- Extract skills from resume using Engine 1 skill extractor
- Extract required_skills from job description using same extractor
      |
      v
Step 2: TF-IDF Vectorization
- Load pre-fitted TfidfVectorizer (fitted on 10,000+ job description corpus)
- Transform resume text -> sparse vector R (shape: 1 x vocab_size)
- Transform job desc -> sparse vector J (shape: 1 x vocab_size)
      |
      v
Step 3: Semantic Similarity
- Compute cosine similarity: sim = dot(R, J) / (norm(R) * norm(J))
- Result: float in [0.0, 1.0]
- semantic_score = round(sim * 100)
      |
```

```

    v
Step 4: Skill Match Bonus
- matched = intersection(resume_skills, job_required_skills)
- missing = job_required_skills - resume_skills
- skill_coverage = len(matched) / max(len(job_required_skills), 1)
- skill_bonus = skill_coverage * 20 (max +20 points)
  |
  v
Step 5: Final Score Calculation
- raw_score = (semantic_score * 0.7) + (skill_bonus * 0.3)
- Wait: no - simpler and more transparent:
- final_score = clamp(semantic_score + skill_bonus, 0, 100)
- final_score = int(final_score) -- always integer
  |
  v
Step 6: Score Breakdown
- score_breakdown = {
    semantic_similarity: semantic_score,
    skill_coverage_pct: round(skill_coverage * 100),
    matched_count: len(matched),
    missing_count: len(missing)
  }
  |
  v
OUTPUT
{
  score: 78,
  matched_skills: [python, machine learning, sql],
  missing_skills: [docker, kubernetes],
  score_breakdown: { semantic_similarity: 65, skill_coverage_pct: 60, ... }
}

```

3.3 Mathematical Foundation

```

TF-IDF Weight for term t in document d:
tfidf(t, d) = tf(t, d) * log(N / df(t))
where:
    tf(t, d) = frequency of term t in document d
    N         = total number of documents in corpus
    df(t)     = number of documents containing term t

Cosine Similarity between vectors R and J:
sim(R, J) = (R . J) / (||R|| * ||J||)
where:
    R . J      = dot product (sum of element-wise products)
    ||R||      = L2 norm of vector R (sqrt of sum of squares)
    ||J||      = L2 norm of vector J

Skill Bonus:

```

```

skill_coverage = |matched_skills| / max(|required_skills|, 1)
skill_bonus    = skill_coverage * 20

Final Score:
final_score = clamp(semantic_score + skill_bonus, 0, 100)
final_score = int(final_score)    -- always integer, never float

Score Bands:
80-100 : Strong Match  (green)
60-79  : Good Match   (teal)
40-59  : Moderate Match (orange)
0-39   : Weak Match   (red)

```

3.4 TF-IDF Model Training

Training Corpus	Minimum 10,000 job descriptions from diverse industries: tech, finance, healthcare, marketing, operations.
Corpus Source	v1.0: Combination of synthetic job descriptions + public datasets (e.g. Kaggle job posting datasets). v1.5: Real platform job postings.
Vectorizer Config	TfidfVectorizer(max_features=50000, ngram_range=(1,3), min_df=2, max_df=0.95, sublinear_tf=True)
max_features	50,000 vocabulary size — balances coverage and memory. Increased to 100,000 in v1.5.
ngram_range (1,3)	Captures unigrams (python), bigrams (machine learning), and trigrams (natural language processing).
min_df=2	Terms appearing in fewer than 2 documents excluded — removes typos and rare noise.
max_df=0.95	Terms appearing in more than 95% of documents excluded — removes stop-word-like terms.
sublinear_tf=True	Applies $\log(1 + \text{tf})$ instead of raw tf — reduces impact of very frequent terms.
Serialization	Fitted vectorizer saved as tfidf_vectorizer.pkl using joblib. Loaded once at AI service startup.
Retraining Trigger	Retrain when: vocabulary coverage drops below 70% on new job postings, or quarterly (whichever comes first).

3.5 Implementation

```

# app/engines/scoring_engine.py
import joblib, numpy as np
from sklearn.metrics.pairwise import cosine_similarity
from pathlib import Path

```

```

from app.engines.resume_engine import extract_skills

# Load at module import - never per-request
VECTORIZER = joblib.load(Path('app/models/tfidf_vectorizer.pkl'))

def score_candidate(resume_text: str, job_description: str,
                    required_skills: list[str] = None) -> dict:

    # Step 1: Vectorize both texts
    vectors = VECTORIZER.transform([resume_text, job_description])
    r_vec    = vectors[0]    # Resume vector
    j_vec    = vectors[1]    # Job description vector

    # Step 2: Compute cosine similarity
    sim = cosine_similarity(r_vec, j_vec)[0][0]
    semantic_score = round(float(sim) * 100)

    # Step 3: Skill gap analysis
    resume_skills = set(extract_skills(resume_text))
    job_skills    = set(required_skills or extract_skills(job_description))

    matched = sorted(resume_skills & job_skills)
    missing = sorted(job_skills - resume_skills)

    # Step 4: Skill coverage bonus (max +20)
    skill_coverage = len(matched) / max(len(job_skills), 1)
    skill_bonus    = round(skill_coverage * 20)

    # Step 5: Final score - clamped to [0, 100]
    final_score = min(100, max(0, semantic_score + skill_bonus))

    return {
        'score': int(final_score),
        'matched_skills': matched,
        'missing_skills': missing,
        'score_breakdown': {
            'semantic_similarity': semantic_score,
            'skill_bonus': skill_bonus,
            'skill_coverage_pct': round(skill_coverage * 100),
            'matched_count': len(matched),
            'missing_count': len(missing),
        }
    }

```

3.6 Scoring Evaluation Benchmark

Metric	Definition	Target
Pearson Correlation	Correlation between system scores and human expert scores on 30 resume-job pairs	Target ≥ 0.80

Score Consistency	Same resume + same job always returns the same score (determinism test)	Target: 100% (zero variance)
Inference Latency	Time from raw text input to JSON output	Target < 500ms at p95
Score Range Validity	All returned scores are integers in [0, 100]	Target: 100% compliance
Skill Precision	% of matched_skills that are genuinely required by the job	Target >= 90%
Skill Recall	% of job required skills that appear in matched or missing (no skills lost)	Target: 100%
Edge Case: Empty Resume	Score for an empty resume string	Target: score = 0, matched_skills = []
Edge Case: Perfect Match	Score when resume text is identical to job description	Target: score = 100

SECTION 4

Engine 3 — Job Recommendation System

4. Engine 3 — Job Recommendation System

The Job Recommendation Engine surfaces the most relevant open positions for each job seeker based on their skill profile. It uses content-based filtering — matching the user vector against all active job vectors — to provide personalized recommendations without requiring any historical interaction data.

4.1 Input / Output Contract

Input	{ user_id: string, skills: string[] } — user ID and their current skills array
Output	{ recommendations: [{ job_id, title, company, location, match_score }] } — top 5 ranked by match_score
Latency SLA	< 1 second (job matrix pre-computed and cached in Redis)
Relevance Target	Click-through relevance $\geq 70\%$ (user clicks at least one recommendation per session)

4.2 Recommendation Algorithm

```
INPUT: user_skills = [python, machine learning, sql, flask]
    |
    v
Step 1: Build User Profile Text
- Join skills into a single string for vectorization
- user_text = ' '.join(user_skills)
- Example: 'python machine learning sql flask'
    |
    v
Step 2: Vectorize User Profile
- Use the SAME TF-IDF vectorizer as scoring engine (consistency)
- user_vector = VECTORIZER.transform([user_text])
- Shape: (1, vocab_size) sparse matrix
    |
    v
Step 3: Load Job Matrix from Redis Cache
- Key: job_matrix:active
- Value: serialized numpy sparse matrix of ALL active job vectors
- Shape: (num_active_jobs, vocab_size)
```

```

- Also load job_ids list: ordered list of job IDs matching matrix rows
- Cache miss: rebuild from DB (see Step 3b)
  |
  v
Step 3b: Rebuild Job Matrix (on cache miss)
- Fetch all active jobs required_skills from DB via Core API
- Build job_texts: [' '.join(job.required_skills) for job in jobs]
- job_matrix = VECTORIZER.transform(job_texts)
- Serialize and store in Redis with TTL 1800s
  |
  v
Step 4: Batch Cosine Similarity
- scores = cosine_similarity(user_vector, job_matrix) -- shape: (1,
num_jobs)
- scores_flat = scores[0] -- 1D array of similarity scores
  |
  v
Step 5: Rank and Select Top 5
- top_indices = np.argsort(scores_flat)[::-1][:5]
- top_job_ids = [job_ids[i] for i in top_indices]
- top_scores = [round(scores_flat[i]*100) for i in top_indices]
  |
  v
Step 6: Fallback (if user has fewer than 3 skills)
- Return 5 most recently posted active jobs
- This ensures new users always see relevant content
  |
  v
OUTPUT
{ recommendations: [
  { job_id: uuid, title: str, match_score: 87 },
  { job_id: uuid, title: str, match_score: 82 },
  ...
]
}

```

4.3 Job Matrix Cache Management

Cache Key	job_matrix:active — stores the serialized numpy sparse matrix and job_ids list
TTL	1800 seconds (30 minutes). Refreshed on every cache hit to keep it warm.
Invalidation Triggers	Cache explicitly deleted when: (1) a new job is created, (2) a job is deleted, (3) a job is closed. Core API calls DELETE /ai/cache/job-matrix on these events.
Serialization	numpy sparse matrix serialized using scipy.sparse.save_npz() to bytes, base64-encoded for Redis storage.
Matrix Size Estimation	10,000 active jobs x 50,000 vocab = 500M float32 values. Stored as sparse matrix: typically < 50MB in memory.

Rebuild Time	Rebuild from DB: < 5 seconds for 10,000 jobs. Triggered only on cache miss — very rare in production.
---------------------	---

4.4 Cold Start Strategy

New User (0 skills)	Return the 5 most recent active jobs. Show a banner: Add skills to your profile to get personalized recommendations.
New User (1-2 skills)	Use the sparse skills vector for recommendation but also supplement with 2 most recent jobs. Flag results as limited personalization.
Established User (3+ skills)	Full recommendation engine. No fallback needed.
New Platform (0 jobs)	Return empty array with message: No jobs available yet. Check back soon.

4.5 Implementation

```
# app/engines/recommendation_engine.py
import numpy as np, json, base64
from scipy.sparse import load_npz, save_npz
from io import BytesIO
from sklearn.metrics.pairwise import cosine_similarity
from app.engines.scoring_engine import VECTORIZER
from app.core.redis import redis_client

MATRIX_CACHE_KEY = 'job_matrix:active'
JOB_IDS_CACHE_KEY = 'job_ids:active'

async def get_job_matrix():
    cached_matrix = await redis_client.get(MATRIX_CACHE_KEY)
    cached_ids = await redis_client.get(JOB_IDS_CACHE_KEY)

    if cached_matrix and cached_ids:
        matrix_bytes = base64.b64decode(cached_matrix)
        job_matrix = load_npz(BytesIO(matrix_bytes))
        job_ids = json.loads(cached_ids)
        return job_matrix, job_ids

    return None, None # Cache miss - caller must rebuild

async def recommend_jobs(user_skills: list[str], user_id: str) -> list[dict]:
    if len(user_skills) < 3:
        return [] # Caller handles fallback to recent jobs

    job_matrix, job_ids = await get_job_matrix()
    if job_matrix is None:
        return [] # Cache miss - return empty, Core API handles fallback
```



```
# Vectorize user skill profile
user_text = ' '.join(user_skills)
user_vector = VECTORIZER.transform([user_text])

# Batch similarity against all jobs
scores = cosine_similarity(user_vector, job_matrix)[0]

# Top 5 by score
top_indices = np.argsort(scores)[::-1][:5]

return [
    {'job_id': job_ids[i], 'match_score': int(round(scores[i] * 100))}
    for i in top_indices
    if scores[i] > 0.0 # Exclude zero-similarity results
]
```

SECTION 5

Engine 4 — AI Interview Evaluation System

5. Engine 4 — AI Interview Evaluation System

The Interview Evaluation Engine scores candidate answers to interview questions and provides structured feedback. Unlike the scoring engine (which uses sparse TF-IDF), the interview engine uses dense sentence embeddings to capture semantic meaning — allowing it to recognise correct answers even when phrased differently from the ideal answer.

5.1 Input / Output Contract

Input	{ question: string, answer: string, domain: string (optional) }
Output	{ score: int (0-100), feedback: string, confidence: low medium high, keywords_found: string[], keywords_missing: string[] }
Latency SLA	< 2 seconds per question-answer pair
Accuracy Target	Human agreement $\geq$ 75% on score band (same band: excellent/good/fair/poor) on 50-pair test set

5.2 Model Selection — Sentence-Transformers

Model	all-MiniLM-L6-v2 (Hugging Face Sentence-Transformers)
Model Size	80MB — lightweight enough for CPU inference within the 2-second SLA
Embedding Dimension	384 dimensions — rich semantic representation
Performance	60,000+ sentences/second on GPU, ~1,500/second on 4-core CPU — more than sufficient
Why Not BERT-base	BERT-base (110M params, 768-dim) is 4x larger and slower. MiniLM-L6 achieves 99% of BERT quality at 25% the size.
Why Not GPT	Generative models are non-deterministic and expensive. Sentence-Transformers are deterministic, fast, and purpose-built for semantic similarity.
Loading	Model loaded once at AI service startup via SentenceTransformer(model_name). Stored in module-level variable.

5.3 Evaluation Pipeline

```
INPUTS: question (Q), candidate_answer (A)
|
v
Step 1: Look Up Ideal Answer
- Load ideal_answers.json corpus
- Find entry matching question (exact match on question hash)
- ideal_answer = corpus[question_hash]['ideal_answer']
- keywords      = corpus[question_hash]['expected_keywords']
- If no entry found: use semantic match against question itself
|
v
Step 2: Edge Case Handling
- If len(answer.strip()) < 20: return score=0, feedback=Too short,
confidence=low
- If answer is copy of question: return score=10, feedback=Did not answer
the question
|
v
Step 3: Encode Both Texts
- answer_embedding = MODEL.encode([candidate_answer]) -- shape: (1, 384)
- ideal_embedding  = MODEL.encode([ideal_answer])      -- shape: (1, 384)
- Both normalised to unit length (normalize_embeddings=True)
|
v
Step 4: Semantic Similarity
- sim = cosine_similarity(answer_embedding, ideal_embedding)[0][0]
- semantic_score = round(sim * 85) -- max 85 from semantics
|
v
Step 5: Keyword Coverage Bonus
- answer_lower = candidate_answer.lower()
- found_keywords = [k for k in keywords if k.lower() in answer_lower]
- missing_keywords = [k for k in keywords if k not in found_keywords]
- keyword_coverage = len(found_keywords) / max(len(keywords), 1)
- keyword_bonus    = round(keyword_coverage * 15) -- max 15 from
keywords
|
v
Step 6: Final Score
- final_score = clamp(semantic_score + keyword_bonus, 0, 100)
- final_score = int(final_score)
|
v
Step 7: Feedback Generation (rule-based)
- score >= 80: Excellent answer. You covered the key concepts clearly.
- score 60-79: Good answer. Consider adding more depth on:
[missing_keywords]
- score 40-59: Partial answer. Key areas to improve: [missing_keywords]
- score < 40: This answer needs significant improvement. Review: [topic]
|
v
Step 8: Confidence Label
```

```

- high    if final_score >= 75
- medium  if final_score >= 50
- low     if final_score < 50
  |
  v
OUTPUT
{
  score:          78,
  feedback:       Good answer. Consider adding more depth on: [SOLID
principles]
  confidence:     medium,
  keywords_found: [OOP, inheritance, polymorphism],
  keywords_missing: [SOLID principles, design patterns]
}

```

5.4 Ideal Answer Corpus

Format	JSON array: [{ id, question, domain, ideal_answer, expected_keywords[], difficulty }]
Size (v1.0)	Minimum 100 questions across 6 domains
Domains	General HR, Frontend Development, Backend Development, Data Science, System Design, Product Management
Question Types	Conceptual (explain X), Behavioral (tell me about a time...), Technical (write code for...), Situational (what would you do if...)
Difficulty Levels	Easy (junior), Medium (mid-level), Hard (senior)
Maintenance	Corpus versioned in Git. New questions added by ML team each sprint. Community contributions reviewed by ML engineer before merge.
Fallback	If question not in corpus: evaluate answer against the question text itself as pseudo-ideal. Accuracy lower but not zero.

5.5 Implementation

```

# app/engines/interview_engine.py
import json, numpy as np
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
from pathlib import Path

# Load model at module import - never per-request
MODEL = SentenceTransformer('all-MiniLM-L6-v2')

with open('app/data/ideal_answers.json') as f:
    raw_corpus = json.load(f)

```

```

# Build lookup by question hash for O(1) retrieval
import hashlib
CORPUS = {
    hashlib.md5(item['question'].lower().encode()).hexdigest(): item
    for item in raw_corpus
}

def evaluate_interview(question: str, answer: str) -> dict:
    # Edge case: answer too short
    if len(answer.strip()) < 20:
        return {'score':0, 'feedback': 'Answer is too short. Please provide a
detailed response.',
                'confidence': 'low', 'keywords_found': [], 'keywords_missing': []}

    # Look up ideal answer
    q_hash = hashlib.md5(question.lower().encode()).hexdigest()
    corpus_entry = CORPUS.get(q_hash)
    ideal = corpus_entry['ideal_answer'] if corpus_entry else question
    keywords = corpus_entry['expected_keywords'] if corpus_entry else []

    # Encode both texts
    embeddings = MODEL.encode([answer, ideal], normalize_embeddings=True)
    a_emb, i_emb = embeddings[0:1], embeddings[1:2]

    # Semantic similarity (max 85 points)
    sim = float(cosine_similarity(a_emb, i_emb)[0][0])
    semantic_score = round(sim * 85)

    # Keyword coverage (max 15 points)
    answer_lower = answer.lower()
    found = [k for k in keywords if k.lower() in answer_lower]
    missing = [k for k in keywords if k not in found]
    kw_bonus = round((len(found) / max(len(keywords), 1)) * 15)

    # Final score
    score = min(100, max(0, semantic_score + kw_bonus))

    # Feedback
    if score >= 80:
        feedback = 'Excellent answer. You covered the key concepts clearly.'
    elif score >= 60:
        feedback = f'Good answer. Consider adding more depth on: {missing[:2]}'
    elif score >= 40:
        feedback = f'Partial answer. Key areas to improve: {missing[:3]}'
    else:
        topic = corpus_entry.get('domain', 'this topic') if corpus_entry else 'this
topic'
        feedback = f'This answer needs significant improvement. Review: {topic}'

    confidence = 'high' if score >= 75 else 'medium' if score >= 50 else 'low'

    return {'score':int(score), 'feedback':feedback, 'confidence':confidence,
            'keywords_found':found, 'keywords_missing':missing}

```

SECTION 6

Model Training Procedures

6. Model Training Procedures

6.1 TF-IDF Vectorizer Training

```
# scripts/train_tfidf.py - Run once to train and save the vectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
import joblib, json, pandas as pd
from pathlib import Path

# Load training corpus
df = pd.read_csv('data/job_descriptions_corpus.csv') # min 10,000 rows
job_texts = df['description'].fillna('').tolist()

# Add required_skills text to corpus for better skill vocabulary coverage
skill_texts = df['required_skills'].fillna('').tolist()
corpus = job_texts + skill_texts

print(f'Training on {len(corpus)} documents...')

vectorizer = TfidfVectorizer(
    max_features = 50000,
    ngram_range = (1, 3),      # Unigrams, bigrams, trigrams
    min_df       = 2,          # Ignore terms in < 2 docs
    max_df       = 0.95,       # Ignore terms in > 95% of docs
    sublinear_tf = True,       # Apply log(1 + tf)
    strip_accents = 'unicode',
    lowercase    = True,
)

vectorizer.fit(corpus)

print(f'Vocabulary size: {len(vectorizer.vocabulary_)}')
print(f'Number of features: {vectorizer.max_features}')

# Save to models directory
output_path = Path('app/models/tfidf_vectorizer.pkl')
joblib.dump(vectorizer, output_path, compress=3)
print(f'Saved to {output_path} ({output_path.stat().st_size / 1024:.1f} KB)')

# Validation: spot-check a few transforms
test_texts = ['python machine learning flask', 'java spring boot microservices']
test_vectors = vectorizer.transform(test_texts)
print(f'Test transform shape: {test_vectors.shape}') # Should be (2, 50000)
assert test_vectors.shape == (2, 50000), 'Shape mismatch!'
print('Training completed successfully')
```

6.2 Training Data Requirements

TF-IDF Corpus	Minimum 10,000 unique job descriptions. Sources: Kaggle job posting datasets, synthetic generation, public job boards (scraped ethically with robots.txt compliance).
Resume Benchmark Set	50 manually annotated resumes for Engine 1 evaluation. Each resume annotated with: correct skills list, experience years, education level. Stored in tests/data/resume_benchmark/.
Scoring Benchmark Set	30 resume-job pairs with human expert scores (0-100) assigned by 2 independent reviewers. Average of both reviewers used as ground truth. Stored in tests/data/scoring_benchmark/.
Interview Corpus	100+ question-ideal_answer pairs. Written by domain experts. Peer-reviewed before addition. Stored in app/data/ideal_answers.json.
Data Privacy	No real user resumes used in training without explicit written consent. Synthetic data preferred. Any consented data anonymised before use.
Data Versioning	All training datasets versioned with DVC (Data Version Control) in the Git repository. Each model version references its training data version.

6.3 Model Versioning Policy

Version	Trigger	Changes & Target
v1.0.0	Initial production model	Trained on synthetic corpus + Kaggle datasets. Baseline benchmark scores established.
v1.1.0	Vocabulary expansion	Retrained with 25,000 job descriptions including real platform postings (anonymised). Target: 5% improvement in scoring correlation.
v1.2.0	Skill dictionary expansion	Skills dictionary grown from 2,000 to 5,000 entries. Resume extraction recall improvement.
v2.0.0	Transformer-based scoring	Replace TF-IDF with SBERT-based scoring using all-mpnet-base-v2. Target: scoring correlation ≥ 0.90 .

SECTION 7

Data Pipeline & Storage

7. Data Pipeline & Storage

7.1 Resume Processing Pipeline

Trigger	User uploads PDF via POST /resumes/upload endpoint in Core API.
Step 1: File Receive	Core API receives file, validates type (PDF) and size (max 5MB). Returns error if invalid.
Step 2: S3 Upload	Valid file uploaded to AWS S3 bucket. File URL stored in resumes.file_url column.
Step 3: Text Extraction	PyMuPDF (fitz) extracts plain text from PDF. Handles multi-column layouts, removes headers/footers.
Step 4: Async AI Trigger	Core API calls FastAPI BackgroundTasks to send text to AI Service /analyze-resume asynchronously. HTTP response returned to user immediately.
Step 5: NLP Processing	AI Service runs full Engine 1 pipeline on the text. Returns structured JSON.
Step 6: DB Update	Core API updates resumes table: parsed_skills, parsed_experience, parsed_education, parsed_projects, parse_status=completed.
Step 7: Cache Update	User skill profile Redis cache invalidated. Job recommendations cache invalidated for this user.

7.2 Application Scoring Pipeline

Trigger	Job seeker submits application via POST /applications.
Step 1: Application Created	Core API creates application record with status=applied, ai_score=null.
Step 2: Async Scoring	BackgroundTask calls AI Service /score-candidate with resume raw_text and job description.
Step 3: Score Computed	Engine 2 runs TF-IDF scoring, returns { score, matched_skills, missing_skills }.
Step 4: DB Update	Core API updates application: ai_score, matched_skills, missing_skills.
Step 5: Cache Store	Score cached in Redis: ai_score:{user_id}:{job_id} with 1-hour TTL.
Failure Handling	If AI Service times out or returns error: application.ai_score remains null. No retry in v1.0. Recruiter sees null score with 'Score pending' label.

7.3 Data Storage Map

Data	Storage Location	Details
Raw resume text	resumes.raw_text	PostgreSQL TEXT column — full text, no truncation
Parsed skills array	resumes.parsed_skills	PostgreSQL TEXT[] array — Engine 1 output
Application AI score	applications.ai_score	PostgreSQL INTEGER — Engine 2 output, null until computed
Matched skills	applications.matched_skills	PostgreSQL TEXT[] — Engine 2 output
Missing skills	applications.missing_skills	PostgreSQL TEXT[] — Engine 2 output
Interview sessions	interview_sessions table	Full Q&A history, per-question scores, overall score
TF-IDF vectorizer	app/models/tfidf_vectorizer.pkl	joblib serialized, loaded at startup
Job matrix	Redis: job_matrix:active	scipy sparse matrix, base64 encoded, TTL 30min
AI score cache	Redis: ai_score:{uid}:{jid}	JSON string, TTL 1hr
Recommendations cache	Redis: job_recs:{user_id}	JSON array, TTL 30min

SECTION 8

ML Service API Contracts

8. ML Service API Endpoint Contracts

8.1 POST /analyze-resume

Method & Path	POST /analyze-resume
Auth	Internal VPC only — not publicly exposed. No JWT required.
Request Body	{ resume_text: string } — raw extracted text from PDF. Max 50,000 characters.
Response 200	{ skills: string[], experience: string null, education: string null, projects: string[], confidence: float }
Response 400	{ error: TEXT_TOO_SHORT } — if resume_text < 50 characters
Response 500	{ error: NLP_PIPELINE_FAILED } — if spaCy processing fails (logged + fallback empty result)
Latency Target	< 2,000ms at p95

8.2 POST /score-candidate

Method & Path	POST /score-candidate
Auth	Internal VPC only — no JWT required.
Request Body	{ resume_text: string, job_description: string, required_skills: string[] (optional) }
Response 200	{ score: int, matched_skills: string[], missing_skills: string[], score_breakdown: { semantic_similarity, skill_bonus, skill_coverage_pct, matched_count, missing_count } }
Response 400	{ error: MISSING_REQUIRED_FIELDS } — if resume_text or job_description is empty
Score Guarantee	score is always an integer in [0, 100]. Never null, never float in production.
Latency Target	< 500ms at p95 (TF-IDF only, no neural network)

8.3 POST /recommend-jobs

Method & Path	POST /recommend-jobs
Auth	Internal VPC only — no JWT required.
Request Body	{ user_id: string, skills: string[] }
Response 200	{ recommendations: [{ job_id: string, match_score: int }] } — max 5 items, ordered by match_score desc
Response (fallback)	{ recommendations: [], fallback: true } — when user has < 3 skills or job matrix is unavailable
Latency Target	< 1,000ms at p95 (job matrix cached in Redis)

8.4 POST /evaluate-interview

Method & Path	POST /evaluate-interview
Auth	Internal VPC only — no JWT required.
Request Body	{ question: string, answer: string, domain: string (optional) }
Response 200	{ score: int, feedback: string, confidence: low medium high, keywords_found: string[], keywords_missing: string[] }
Response 400	{ error: ANSWER_TOO_SHORT } — if answer < 20 characters
score Guarantee	Always integer in [0, 100].
Latency Target	< 2,000ms at p95 (sentence transformer encoding is the bottleneck)

8.5 GET /ai/health

Method & Path	GET /health
Auth	Public — used by load balancer health checks
Response 200	{ status: ok, models_loaded: { spacy: true, tfidf: true, sentence_transformer: true }, latency_ms: int }
Response 503	{ status: degraded, models_loaded: { spacy: false, ... } } — if any model failed to load at startup
Startup Behaviour	Service returns 503 until ALL models are loaded. Load balancer holds traffic until healthy.

SECTION 9

ML Testing & Evaluation Framework

9. ML Testing & Evaluation Framework

9.1 Test Suite Structure

```
ai_service/tests/
  conftest.py           # Shared fixtures: sample resumes, sample jobs
  test_resume_engine.py # Unit tests for Engine 1 (NLP parsing)
  test_scoring_engine.py # Unit tests for Engine 2 (TF-IDF scoring)
  test_recommendation_engine.py # Unit tests for Engine 3 (content-based recs)
  test_interview_engine.py # Unit tests for Engine 4 (semantic eval)
  test_api_routes.py    # Integration tests for all 4 FastAPI endpoints
  benchmarks/
    run_resume_benchmark.py # Runs Engine 1 against 50-resume gold set
    run_scoring_benchmark.py # Runs Engine 2 against 30-pair human scores
    run_interview_benchmark.py # Runs Engine 4 against 50-pair human ratings
  data/
    resume_benchmark/      # 50 annotated PDF text files + labels JSON
    scoring_benchmark/     # 30 resume-job pairs + human scores CSV
    interview_benchmark/   # 50 Q&A pairs + human rating scores
    sample_resumes/        # Small set of realistic resume texts for unit
  tests
```

9.2 Engine 1 Unit Tests

Test Name	Description	Priority
test_extract_python_skill	Resume containing Python developer. Expects python in output skills.	REQUIRED
test_extract_multiple_skills	Resume with Python, React, Docker, PostgreSQL. All 4 extracted.	REQUIRED
test_skill_case_insensitive	PYTHON, Python, python all map to same canonical skill.	REQUIRED
test_skill_alias	ML, Machine Learning, machine-learning all map to machine learning.	REQUIRED
test_experience_date_range	Resume with 2019-2022 experience. Returns 3 years.	REQUIRED
test_experience_present	Resume with 2020-Present. Returns 4+ years (relative to current year).	REQUIRED

test_education_btech	Resume with B.Tech Computer Science. Education correctly extracted.	REQUIRED
test_empty_resume	Empty string input. Returns empty skills[], null experience/education.	REQUIRED
test_noise_text	Resume with URLs, emails, phone numbers. Noise removed before processing.	REQUIRED
test_projects_section	Resume with Projects section. Project names extracted.	REQUIRED

9.3 Engine 2 Unit Tests

Test Name	Description	Priority
test_score_is_integer	Score output is always int, never float.	REQUIRED
test_score_range	Score always in [0, 100] for any valid input.	REQUIRED
test_perfect_match	Resume text == job description text. Score = 100.	REQUIRED
test_no_match	Completely unrelated texts. Score < 30.	REQUIRED
test_matched_skills_subset	matched_skills is always a subset of resume skills.	REQUIRED
test_missing_skills_subset	missing_skills is always a subset of job required_skills.	REQUIRED
test_deterministic	Same inputs always return identical output.	REQUIRED
test_empty_resume_text	Empty resume. Score = 0.	REQUIRED
test_score_breakdown_present	Output always contains score_breakdown object with expected keys.	REQUIRED
test_latency	Single scoring request completes in < 500ms.	REQUIRED

9.4 Benchmark Evaluation Procedure

Resume Benchmark	Run python benchmarks/run_resume_benchmark.py. Loads 50 annotated resumes. Runs Engine 1 on each. Computes precision and recall vs gold labels. Prints per-resume breakdown + aggregate metrics. Target: recall >= 85%, precision >= 90%.
Scoring Benchmark	Run python benchmarks/run_scoring_benchmark.py. Loads 30 resume-job pairs + human scores. Runs Engine 2 on each pair. Computes Pearson correlation between system scores and human scores. Target: correlation >= 0.80.

Interview Benchmark	Run python benchmarks/run_interview_benchmark.py. Loads 50 Q&A pairs + human ratings (1-4 scale). Maps system score to same 4-level scale. Computes % agreement on score band. Target: band agreement >= 75%.
When to Run	Before every model version release. Automated in CI pipeline (benchmarks run on merge to model-release branch). Results committed to benchmarks/results/ directory.
Regression Gate	If any benchmark score drops below target vs previous version: new model is BLOCKED from deployment. No exceptions.

SECTION 10

Model Deployment & Serving

10. Model Deployment & Serving

10.1 Model Loading Strategy

```
# app/core/model_loader.py - Load all models at startup
import spacy, joblib
from sentence_transformers import SentenceTransformer
from pathlib import Path
import structlog

logger = structlog.get_logger()

class ModelRegistry:
    spacy_model:      any = None
    tfidf_vectorizer: any = None
    sentence_model:   any = None
    models_loaded:    bool = False

MODELS = ModelRegistry()

def load_all_models():
    logger.info('Loading ML models...')

    # 1. spaCy model (~750MB on first download, cached locally)
    logger.info('Loading spaCy en_core_web_lg...')
    MODELS.spacy_model = spacy.load('en_core_web_lg')
    logger.info('spaCy loaded')

    # 2. TF-IDF vectorizer (~5MB serialized)
    logger.info('Loading TF-IDF vectorizer...')
    MODELS.tfidf_vectorizer = joblib.load(Path('app/models/tfidf_vectorizer.pkl'))
    logger.info('TF-IDF loaded')

    # 3. Sentence Transformer (~80MB)
    logger.info('Loading Sentence Transformer...')
    MODELS.sentence_model = SentenceTransformer('all-MiniLM-L6-v2')
    logger.info('Sentence Transformer loaded')

    MODELS.models_loaded = True
    logger.info('All models loaded successfully.')

# FastAPI lifespan - called before first request
from contextlib import asynccontextmanager
from fastapi import FastAPI
```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    load_all_models() # Blocking — ensures all models ready before serving
    yield             # App serves requests
    logger.info('AI Service shutting down.')
```

10.2 Deployment Configuration

Containerization	AI Service runs in its own Docker container. Dockerfile uses python:3.11-slim base. Models downloaded during image build (not at runtime).
Model Baking	spaCy model downloaded via <code>RUN python -m spacy download en_core_web_lg</code> in Dockerfile. Sentence Transformer pre-downloaded via <code>RUN python -c 'from sentence_transformers import SentenceTransformer; SentenceTransformer(model_name)'</code> .
Cold Start	Because models are baked into the Docker image, cold start is 10-15 seconds (model loading from disk). ECS keeps at least 1 task running at all times.
Scaling	AI Service scales independently of Core API. Scale-out trigger: CPU > 70% or latency p95 > 4s. Min tasks: 1. Max tasks: 5 (v1.0).
Resource Sizing	ECS Task: 2 vCPU, 4GB RAM. spaCy: ~1.5GB, SentenceTransformer: ~500MB, TF-IDF: ~200MB. Total: ~2.2GB — fits comfortably in 4GB.
GPU	Not required for v1.0. All inference on CPU. GPU added in v2.0 when transformer-based scoring is introduced.
Health Check	ALB health check: GET /health every 30 seconds. Task replaced if health check fails 3 consecutive times.

10.3 Model Update Procedure

Step 1 — Train	Run training script on updated corpus. New model saved as <code>tfidf_vectorizer_v{version}.pkl</code> .
Step 2 — Evaluate	Run all 3 benchmark scripts. All benchmarks must meet or exceed targets vs previous model version.
Step 3 — PR	Open PR to model-release branch. Include benchmark comparison report. Requires ML engineer + Tech Lead approval.
Step 4 — Build	CI builds new Docker image with updated model baked in. Tags image as <code>ai-service:v{version}</code> .
Step 5 — Staging	Deploy new image to staging. Run smoke tests + manual QA with 5-10 real resumes and job descriptions.
Step 6 — Production	Blue-green deployment on ECS. New task set starts, health checks pass, traffic switched, old task set terminated.

Step 7 — Monitor	Monitor scoring distribution for 24 hours post-deploy. Alert if average score drops or spikes > 15% vs baseline.
Rollback	If any alert fires within 24 hours: rollback to previous image tag via single ECS task definition update. < 2 minutes.

SECTION 11

ML Monitoring & Drift Detection

11. ML Monitoring & Drift Detection

11.1 Production ML Metrics

Metric	Description	Alert Condition
Score Distribution Mean	Average AI score across all applications in 24hr window	Alert if mean drifts > 15 points from 7-day rolling average
Score Distribution Std Dev	Spread of AI scores	Alert if std dev drops below 5 (scores clustering — model may be degenerate)
Null Score Rate	% of applications where ai_score is null (AI failed)	Alert if > 5% of applications have null score in 1hr window
Inference Latency p95	95th percentile response time for AI endpoints	Alert if > 4,000ms (below 5s SLA with buffer)
Recommendation Click-Through	% of recommended jobs that user clicks	Alert if < 40% — indicates recommendation quality degradation
Interview Score Distribution	Distribution of interview scores across all sessions	Alert if > 60% of scores are > 90 (may indicate corpus over-fitting)

11.2 Data Drift Detection

What is Data Drift	When the statistical distribution of new incoming data (job descriptions, resumes) shifts significantly from the distribution the model was trained on. Causes degraded model performance without obvious errors.
Detection Method	Monitor vocabulary coverage: % of unique tokens in new job descriptions that exist in the TF-IDF vocabulary. Alert if coverage drops below 70%.
Monitoring Frequency	Weekly automated script runs on a sample of the last 1,000 job descriptions. Results logged to CloudWatch.
Retraining Trigger	If vocabulary coverage < 70%, OR quarterly — whichever comes first.
Resume Drift	Monitor average skill count per resume over time. Alert if average drops below 5 (may indicate resume format changes that confuse the parser).

11.3 Bias Monitoring

Bias Principle	HireIQ scoring must be based exclusively on skill and semantic match. No demographic signals should influence scores.
Feature Audit	Quarterly audit: verify that no name, location, graduation year, or other demographic signals are present in the text passed to the scoring engine. Text cleaning step must remove all personal identifiers.
Score Fairness Check	Monthly: compare average AI scores for job seekers grouped by experience level (fresh grad, 1-3yr, 5yr+). Alert if any group has average score > 20 points above another group for identical job descriptions.
Skills-Only Guarantee	The TF-IDF vectorizer is trained on job descriptions only — it has no demographic vocabulary. A resume with only skills and experience terms produces the same score as one with demographic information added.
Audit Log	All scoring requests logged (not stored): input text hash, output score. Allows retrospective auditing without storing PII.

SECTION 12

ML Roadmap & Future Enhancements

12. ML Roadmap & Future Enhancements

12.1 v1.0 (Current) vs v2.0 (Planned)

Component	v1.0 Approach	v2.0 Planned Upgrade
Resume Parsing	spaCy NER + custom skill dictionary. Rule-based extraction.	Fine-tuned BERT NER model trained on 10,000 labelled resume segments. Higher entity coverage and accuracy.
Candidate Scoring	TF-IDF + cosine similarity. Explainable, fast, CPU-only.	SBERT (all-mpnet-base-v2) dense embeddings. Semantic understanding without exact keyword match. Requires GPU.
Job Recommendations	Content-based TF-IDF similarity. No user history needed.	Hybrid: content-based + collaborative filtering. Learns from click patterns and application outcomes.
Interview Evaluation	Sentence-Transformers + rule-based feedback.	Fine-tuned LLM (7B param, quantized) for nuanced, question-specific feedback generation.
Bias Detection	Manual quarterly audit.	Automated counterfactual bias testing: submit identical resumes with varied demographic signals, verify score invariance.
Explainability	matched_skills + missing_skills arrays.	SHAP values showing feature importance for each score. Visual explanation: which parts of the resume contributed most.

12.2 Research Experiments Backlog

Experiment 1	Fine-tune a BERT-based NER model on a labelled resume dataset (ResumeNER or synthetic). Compare skill recall vs current spaCy + dictionary approach. Success criteria: recall improvement > 5%.
Experiment 2	Replace TF-IDF scoring with SBERT all-mpnet-base-v2 semantic similarity. Benchmark against current TF-IDF on 30-pair human score set. Success criteria: Pearson correlation improvement > 0.05.

Experiment 3	Add collaborative filtering layer to job recommendations using implicit feedback (views, applications, time-on-page). Success criteria: click-through rate improvement > 15% in A/B test.
Experiment 4	Test GPT-4o-mini for interview feedback generation (constrained by prompt to be deterministic). Compare feedback quality ratings vs current rule-based approach. Success criteria: user rating improvement > 0.5 on 1-5 scale.
Experiment 5	Personality trait analysis from interview text answers (Big Five). Validate against standardised psychometric tests. NOTE: Only implemented if bias audit passes.

HireIQ MLSD v1.0 — End of Document

This ML Specification Document defines all AI models, data pipelines, training procedures, evaluation benchmarks, and deployment standards for HireIQ v1.0.